LedgerLens: Month-End Evidence Review

Ayaan Khan | MiraFlores Case Study | Prepared for GreenGrowth CPAs

Live prototype: https://ayaan-27.github.io/Miraflores_Project/

Source code and README: https://github.com/ayaan-27/Miraflores_Project

1. The problems I considered and why I chose this one

Supporting evidence is difficult to connect to the work it supports. At Ricoh, I worked on a document-retrieval assistant that preserved references between service instructions and their supporting diagrams. That experience motivated me to look for other workflows where finding an answer is only part of the task; someone must also verify the evidence behind it. In an accounting context, I was interested in the relationship between bank transactions, receipts, and the reviewer's next action. My hypothesis is that fragmented evidence creates repeated searches and client requests for bookkeeping teams, while clients experience repeated questions and a slower close.

Operational users struggle to get reliable answers across databases. I have worked on a text-to-SQL system at Ricoh spanning more than 50 relational databases and a curated knowledge graph. That work motivated my interest in how data access and business definitions affect the usefulness of an answer. Operational users can depend on specialists to retrieve information or interpret results. If those barriers remain, decisions take longer and different teams may work with inconsistent numbers. I considered building a narrowly scoped analytics assistant to address this problem.

Model builders need useful data without exposing sensitive information. At Mphasis, I worked on synthetic-data generation and sensitive-information redaction. Those projects motivated my interest in the tradeoff between making data available for development and protecting the people or organizations represented in it. Model builders and data owners both experience this tension: restricted access can slow evaluation, while uncontrolled sharing creates privacy risks. I considered a tool for preparing safer evaluation datasets.

I chose the first problem because it connected my experience with traceable information systems to a specific workflow I could demonstrate end to end. The other two would require validated database semantics or a substantial privacy-and-utility evaluation. Evidence review offered a smaller experiment: take a transaction, identify plausible support, explain uncertainty, and produce a useful next action.

I have not interviewed GreenGrowth's staff or observed its internal process. My previous work informs the design, but the frequency and cost of this accounting problem are assumptions to validate. All prototype data is synthetic.

2. How I thought about the problem and developed the approach

I started by asking where a professional loses time turning available information into a decision they can defend. My experience with document retrieval and analytics made supporting

evidence a natural focus. For this project, I narrowed that question to the handoff between a bank transaction, a receipt, and a review decision.

I broke the handoff into three questions: Is supporting evidence available? Is the relationship unambiguous? What should happen next? A receipt with the same amount is not enough. Two transactions may compete for it, merchant names may differ, or evidence may be missing. This led to the main design principle: explain uncertainty and give the reviewer a next step.

My working assumption is that reviewers currently combine transaction exports, receipt folders or spreadsheets, and client emails. Existing accounting tools may already solve parts of this workflow. A better outcome would be less searching, a clearer exception queue, and more specific requests, with professional judgment remaining with the reviewer.

The smallest useful first step. I chose two structured CSV inputs: bank transactions and a receipt index. The prototype identifies candidate matches, explains exceptions, records a reviewer decision, and exports a review register and an editable client request. This tests the decision loop before adding document extraction or integrations. It does not perform bank balance reconciliation, determine tax treatment, or post accounting entries.

The alternative I considered. A standardized spreadsheet and client-request checklist could be cheaper to adopt and sufficient at low volume. I chose a browser application to test whether consistent matching rules, receipt-allocation constraints, and source references improve that process. If a spreadsheet performs equally well, a separate product would be difficult to justify. Adding this workflow to an existing accounting platform is another possible direction after understanding its current capabilities.

Where AI belongs. The core comparisons are exact signed amounts, dates, merchant names, and allocation constraints. I used explicit rules because they are repeatable and easy to inspect. An LLM-generated confidence score would not improve those financial checks. AI could later help extract receipt fields or suggest merchant aliases, but those suggestions would need source references, measured error rates, and human verification.

The broader opportunity is a shared review workspace across clients, connected to existing bookkeeping systems. I would pursue that only after establishing that the exception queue and next-action workflow save enough effort to be useful.

3. The working prototype

LedgerLens loads a fictional client with 12 bank transactions and 10 receipt-index records. Users can also import their own CSV pair. The application runs entirely in the browser, with no backend, paid service, account, or model API key.

Matching logic. A receipt candidate must have the same signed USD amount, a date within the selected tolerance (three days by default), and at least 50% overlap between normalized merchant-name tokens. Amounts are compared as integer cents. Only a unique exact merchant match with no competing transaction is marked matched automatically. Partial names, multiple candidates, shared receipts, and duplicate bank records require review. An item with no qualifying candidate is marked as missing evidence.

Reviewer decisions and outputs. A reviewer can confirm a candidate, exclude an item with an explanation, or mark evidence requested. Each decision requires a note. One receipt cannot be allocated to two transactions, and requesting evidence keeps the item open. The original matching explanation is retained. The reviewer can download a CSV register, a JSON snapshot, and an editable client-request draft listing unresolved items. No email is sent automatically.

Representative use case. Blue Bottle Coffee, transaction B004 for $48.50, has one same-day receipt indexed as Blue Bottle. The partial name match requires review. For the synthetic demonstration, the reviewer adds a note and confirms R004. The summary changes from six of twelve items accounted for and $593.50 unresolved to seven accounted for and $545.00 unresolved. In actual use, the reviewer would first inspect the underlying receipt separately; this prototype only reads its metadata.

Difficult and invalid-input cases. Staples transactions B005 and B006 both qualify for receipt R005. Confirming it for one causes a second assignment to be rejected. Harbor Print has two eligible receipts and remains ambiguous. A negative $45 credit memo demonstrates refund matching. Importing the supplied invalid bank file fails on duplicate ID B001 without erasing the current dataset or decisions.

Implemented, synthetic, and deferred. CSV validation, matching, explanations, search, date tolerance, reviewer notes, undo, allocation constraints, and downloads are implemented. The client, transactions, receipt records, and filenames are synthetic; actual PDFs and images are not provided. Document upload, OCR, authenticated users, persistent history, accounting and email integrations, foreign currencies, and split or combined payments are deferred.

The matching engine is separate from the interface to make its decisions testable. Imported data remains in page memory and clears on reload, so users must export before leaving. The JSON snapshot is an editable record, not a tamper-evident audit log or a resumable session.

4. Validation, next steps, and use of AI

What was checked. All 13 engine tests passed, covering expected sample outputs, cents arithmetic, date boundaries, CSV quoting, ambiguous and duplicate records, refunds, invalid inputs, decision constraints, and export protections. Browser acceptance checks exercised actual imports, decisions, downloads, receipt-reuse rejection, invalid-import preservation, mobile layout, and keyboard dialog dismissal. Those runs produced no page errors or unexpected external requests. The production build passed, and the deployment workflow verified the public application assets against the tested build. These checks establish prototype behavior; real-world matching accuracy still needs evaluation.

What I would learn next. I would first interview three bookkeeping reviewers and inspect consented, redacted examples of their current process. I want to establish whether the bottleneck is finding evidence, understanding business purpose, preparing requests, or waiting for clients. I would then compare LedgerLens with a standardized spreadsheet using five to ten willing client datasets.

The measures would be reviewer minutes per 100 transactions, request completeness, repeated requests, and incorrect evidence assignments, adjudicated by a reviewer. Synthetic

examples cannot establish those outcomes. My provisional pilot goal is at least 25% less reviewer time without increasing incorrect assignments; that is a target to test, not an achieved saving.

When I would stop or change direction. I would narrow or stop the product if existing tools already solve the problem, matching is not a meaningful source of effort, preparing the receipt index takes more time than the workflow saves, or the spreadsheet performs just as well. Incorrect automatic assignments would require investigating and tightening the matching rules.

If the workflow proves useful, I would next investigate reviewed document extraction to reduce index preparation, then authenticated sessions and durable decision history. Integrations and appropriate access controls would precede a trial with real client information. Support for split payments would depend on how often that case occurs.

AI assistance and reused work. I used an AI coding assistant to help develop the scope, interface, matching implementation, tests, and written draft. The generated implementation was checked through automated tests and browser acceptance checks, including conflicting decisions and invalid inputs. No employer code, client records, or prior application code was reused. The product's matching logic uses conventional software; it does not call an AI model at runtime.

Time and deliverables. The initial AI-assisted build, writing, testing, and packaging took approximately 25 minutes, followed by revisions to the submission and video script. That estimate excludes my personal review and video recording. The live prototype, source code, README, synthetic samples, and expected outputs are linked above. My next investigation would be the reviewer interviews and spreadsheet comparison described here.